

Практическая работа 8

Единица эволюции и контекст

Единица эволюции – Сбор документов (Collection).

- API: POST /v1/collections (Создание сбора).
- Схема/Таблица: collections.

Контекст:

- Доля старых клиентов (C1): 70%. (Мы должны поддерживать их долго).
- Требования: Миграция данных без даунтайма; поддержка старых клиентов (C1) на новых серверах (S2) в режиме обратной совместимости.

Паспорт контракта v1

API (Запрос POST /v1/collections v1):

- company_id (uuid, Обязательно)
- counterparty_id (uuid, Опционально)
- type (string, Обязательно, Enum)
- documents (array, Обязательно)
- pushing (string, Опционально, Enum: EMAIL, SMS, PUSH)
- expired_date (string, Опционально, RFC3339)

Схема (Таблица collections v1):

- id (uuid, PK)
- company_id (varchar, R)
- status (varchar, R)
- pushing (varchar, R, Enum)
- verification_template (varchar, R)
- expired_date (timestampz, R)

Смысловые инварианты (v1):

- Стратегия уведомления (pushing) — это простое строковое перечисление (EMAIL, SMS, PUSH).
- Сервер сам определяет verification_template

Проектирование v2 (Эволюция)

Причина изменений:

1. (Аддитивная) Бизнес хочет, чтобы бухгалтеры могли оставлять внутренние заметки к сбору, которые не видит клиент.
2. (Ломающая) Бизнес хочет, чтобы стратегии уведомлений (pushing) были не просто ENUM, а сложными, настраиваемыми сущностями (например, "EMAIL, если нет ответа 3 дня, то SMS").

Изменение 1 (Аддитивное/Совместимое):

- API: Добавляем internal_memo (string, Опционально) в запрос POST /v2/collections.
- DB: Добавляем столбец internal_memo (text, nullable) в таблицу collections.

Изменение 2 (Ломающее):

- API: Поле pushing (string) в POST /v2/collections заменяется на pushing_strategy_id (uuid, Опционально).
- DB: Столбец pushing (varchar) заменяется на pushing_strategy_id (uuid, FK, nullable).

ER-diff (v1 ↔ v2)

Поле	v1 (POST /v1/collections)	v2 (POST /v2/collections)	Тип изменения
company_id	uuid (Обязательно)	uuid (Обязательно)	untouched
counterparty_id	uuid (Опционально)	uuid (Опционально)	untouched
type	string (Обязательно, Enum)	string (Обязательно, Enum)	untouched
documents	array (Обязательно)	array (Обязательно)	untouched
expired_date	string (Опционально)	string (Опционально)	untouched
pushing	string (Опционально, Enum)	Нет	Ломающее
pushing_strategy_id	(Отсутствует)	uuid (Опционально)	Ломающее (Замена pushing)
internal_memo	(Отсутствует)	string (Опционально)	Аддитивное

collections			
uuid	id	PK	
varchar_255	company_id		NOT NULL
enum	status		NOT NULL
varchar_50	pushing		NOT NULL
varchar_100	verification_template		NOT NULL
timestampz	expired_date		NOT NULL

collections_v2			
uuid	id	PK	
varchar_255	company_id		NOT NULL
enum	status		NOT NULL
varchar_100	verification_template		NOT NULL
timestampz	expired_date		NOT NULL
text	internal_memo		O (Additive)
uuid	pushing_strategy_id		O, FK (Breaking)



pushing_strategies			
uuid	id	PK	
varchar	name		R
jsonb	rules		R

Матрица совместимости «Client×Server»

- C1: Старый клиент, использует /v1 , отправляет pushing: "EMAIL".
- C2: Новый клиент, использует /v2, отправляет pushing_strategy_id: "uuid-...".
- S1: Старый сервер, знает только о /v1.
- S2: Новый сервер, поддерживает и /v1 (для C1) и /v2 (для C2).

Клиент \ Сервер	S1 (Только /v1)	S2 (Поддерживает /v1 и /v2)
C1	C1 шлет pushing: "EMAIL". S1 сохраняет "EMAIL".	C1 шлет pushing: "EMAIL" на /v1. S2 на эндпоинте /v1 преобразует "EMAIL" в pushing_strategy_id: "uuid-email" и сохраняет. internal_memo = NULL.
C2	S1 не имеет эндпоинта /v2. Сервер отвечает 404 Not Found.	C2 шлет pushing_strategy_id и internal_memo на /v2. S2 всё сохраняет.
C1	S1 не имеет эндпоинта /v2.	C1 шлет pushing: "EMAIL" на /v2. S2 (v2) ожидал pushing_strategy_id. Сервер отвечает 422 Unprocessable Entity.

Правила Negotiation и Деградаций (Fallback)

Negotiation (Согласование): Основное согласование происходит через версионирование URL (/v1 или /v2). Клиенты C2 также должны отправлять заголовок X-Client-Version: 2 для метрик.

Fallback (Деградация): Режим Degraded (Fallback) (ячейка C1 -> S2) — самый важный.

- Правило: когда C1 отправляет pushing: "EMAIL" на эндпоинт /v1 сервера S2, S2 *обязан* это принять.
- Действие: S2 смотрит на свой pushing_strategy_mapping ('EMAIL' -> 'uuid-email') и подменяет данные.
- Почему это лучше ошибки: Мы сохраняем обратную совместимость. Клиент v1 не знает о v2, но его *намерение* (отправить email) выполняется.

Политика Депрекейта (Deprecation)

Мы используем модель "Анонс -> Двойная работа -> Отключение".

1. Announce (Анонс) (День 0):
 - Запускаем S2 с поддержкой /v2.
 - Сервер S2 на запросы /v1 начинает отдавать заголовки: Deprecation: true и Sunset: <Today+90 days>.
2. Dual-run (Двойная работа) (Дни 0-120):
 - Поддерживаем оба API (/v1 и /v2).
 - Условие: Мониторим дашборд: "Доля трафика на /v1".
 - План отката: Если на /v2 всплеск ошибок 5xx, откатываем S2 на S1.
3. Stop-write-old (Отключение записи) (День 120):
 - Условие: Если трафик /v1 < 2%.
 - Эндпоинты POST и DELETE на /v1 начинают возвращать 410 Gone. (GET еще работает).
4. Stop-read-old (Полное отключение) (День 180):

- Отключаем /v1 (возвращает 410 Gone на все).

Миграция данных (Expand → Migrate → Contract)

Миграция pushing (varchar) на pushing_strategy_id (uuid) без даунтайма:

1. Expand (Deploy 1):
 - Добавляем таблицу pushing_strategies (с 'uuid-email' и т.д.).
 - Добавляем столбец collections.pushing_strategy_id (nullable).
 - Выкатываем S2-код, который при *записи* в /v1 (получая pushing: "EMAIL") записывает в оба столбца: pushing = "EMAIL" и pushing_strategy_id = "uuid-email".
2. Migrate (Async):
 - Запускаем скрипт, который в фоне обновляет все старые записи: UPDATE collections SET pushing_strategy_id = ... WHERE pushing IS NOT NULL AND pushing_strategy_id IS NULL.
3. Contract (Deploy 2):
 - Выкатываем S2-код, который при *чтении* из /v1 *перестает* смотреть на столбец pushing и читает только из pushing_strategy_id (конвертируя "uuid-email" обратно в "EMAIL" для ответа C1).
4. Contract (Deploy 3 - после Депрекейта):
 - Удаляем столбец collections.pushing.

Наблюдаемость

Обязательные метки (Labels):

- api_version (из URL: v1 или v2).
- client_version (из заголовка X-Client-Version).

Сигналы деградации (Alerts):

- Alert 1: rate(422_errors{api_version="v2"}) > 0 — Сигнал: C1 пытается использовать v2.
- Alert 2: rate(traffic{api_version="v1"}) — Дашборд: Мониторинг прогресса депрекейта.

- Alert 3: $\text{rate}(5xx_errors\{api_version="v2"\}) > \text{rate}(5xx_errors\{api_version="v1"\})$ — Сигнал: v2 менее стабилен, чем v1 (нужен откат).

Каталог проверок (Сценарии)

1. Сценарий: C1 -> S1 (/v1)
 - Запрос: POST /v1/collections, {"pushing": "EMAIL"}
 - Ожидание: 201 Created. В БД pushing = "EMAIL". (Штатная работа v1).
2. Сценарий: C2 -> S2 (/v2)
 - Запрос: POST /v2/collections, {"pushing_strategy_id": "uuid-email", "internal_memo": "note"}
 - Ожидание: 201 Created. В БД pushing_strategy_id = "uuid-email", internal_memo = "note". (Штатная работа v2).
3. Сценарий: C1 -> S2 (/v1) (Деградация)
 - Запрос: POST /v1/collections, {"pushing": "EMAIL"}
 - Ожидание: 201 Created. S2 преобразовал данные. В БД pushing_strategy_id = "uuid-email", internal_memo = NULL. (Обратная совместимость S2 работает).
4. Сценарий: C2 -> S1 (/v1) (Деградация)
 - Запрос: C2 (умный клиент) видит, что S1 (v1). Он сам выполняет преобразование. POST /v1/collections, {"pushing": "EMAIL"}.
 - Ожидание: 201 Created. S1 сохранил pushing = "EMAIL". (Обратная совместимость C2 работает).
5. Сценарий: C1 -> S2 (/v2) (Блокировка)
 - Запрос: C1 (старый клиент) ошибочно шлет на /v2. POST /v2/collections, {"pushing": "EMAIL"}.
 - Ожидание: 422 Unprocessable Entity. Сообщение: field: pushing_strategy_id, message: is required (или "pushing is unknown field"). (S2 корректно заблокировал невалидную схему v1).
6. Сценарий: C2 -> S1 (/v2) (Блокировка)

- Запрос: C2 шлет на /v2.
- Ожидание: 404 Not Found. (S1 не знает о v2).

7. Сценарий: Ретрай C1 -> S2 (/v1)

- Запрос: C1 шлет POST /v1/collections с Idempotency-Key: K1 и pushing: "EMAIL". S2 отвечает 201. C1 повторяет запрос (сеть моргнула).
- Ожидание: 201 Created (из кэша идемпотентности). S2 *не* создает второй сбор. (Идемпотентность работает сквозь версии).